

Project Specification: Real-Time Stress & Recovery Tracker

This document serves as the master blueprint for an AI coding assistant (like GitHub Copilot for Xcode) to generate a SwiftUI-based iOS application that tracks stress, energy, and recovery with high precision.

1. Architecture & Tech Stack

- **Framework:** SwiftUI (Targeting iOS 17+)
- **Data Source:** HealthKit (HKHealthStore)
- **Concurrency:** Swift async/await
- **Local Storage:** SwiftData (for caching baseline metrics and daily timelines)
- **Charting:** Swift Charts framework

2. The "High Accuracy" Data Ingestion Strategy

To match the accuracy of top-tier apps like Livity, the app must bypass the default Apple Watch Heart Rate Variability (HRV) sampling rate. By default, the watch only samples HRV 3 to 5 times a day, which makes real-time stress tracking impossible. Copilot must be instructed to include an onboarding screen that prompts the user to enable **AFib History** in the Apple Health app. Enabling this forces the Apple Watch to log an HRV reading every 5 to 15 minutes, feeding the app a continuous stream of data.

HealthKit Queries Required

1. `HKObserverQuery`: Listen for new HRV (SDNN) and Heart Rate samples in the background.
2. `HKStatisticsCollectionQuery`: Fetch 30-day rolling baselines for HRV and Resting Heart Rate (RHR) to establish the user's unique baseline.
3. `HKSampleQuery`: Fetch the last 24 hours of HRV data to construct the daily timeline chart.

3. Mathematical Algorithms

The app requires a local computational engine to convert raw HealthKit data into normalized 0-100 scores.

The Stress Algorithm (Z-Score Normalization)

Stress is calculated inversely to HRV deviations from the user's personal baseline. Copilot must implement the following logic inside an `AlgorithmEngine` service:

- Calculate baseline mean (μ) and standard deviation (σ) from the last 30 days of HRV.

- Calculate Z-Score for incoming data: $(\text{Current HRV} - \mu) / \sigma$
- Map to 0-100 Scale: $50 - (\text{Z-Score} * 15)$
- Clamp values firmly between 0 and 100.

Stress Level Tiers

The UI must adapt based on these five distinct physiological states:

Score Range	State	Color Palette	Physiological Meaning
0 - 20	Zen / Deep Rest	Deep Blue / Teal	Parasympathetic dominance. Deep physiological recovery.
21 - 40	Relaxed	Light Blue / Mint	Calm baseline state. Standard resting operations.
41 - 60	Balanced	Yellow / Gold	Active but not overwhelmed. Mild exertion or focus.
61 - 80	Strained	Orange	Sympathetic dominance. High cognitive or physical load.
81 - 100	Overwhelmed	Crimson / Red	Peak stress. Immediate rest or breathwork recommended.

4. UI/UX & Animation Specifications

The interface must feel organic, fluid, and alive, heavily relying on SwiftUI's advanced rendering capabilities rather than static assets.

The Main Dashboard

- **The Energy Battery (Hero Element):** A large circular gauge that dynamically fills based on the daily energy score. Copilot must use a `TimelineView` combined with an `AngularGradient` to create a slow, pulsing rotation effect along the ring's stroke, making it look like flowing energy.
- **The Stress Wave (Live Mood Animation):** Copilot should implement a `Canvas` view containing overlapping sine waves. The frequency, amplitude, and color of the waves must bind directly to the current Stress Score. For example, a high stress score triggers fast, jagged red waves, while a low score triggers slow, rolling blue waves.
- **Timeline Chart:** A smooth `Swift Charts AreaMark` graph showing the day's stress fluctuations. Apply `.interpolationMethod(.catmullRom)` to make the line perfectly smooth and organic rather than a jagged, mathematical line graph. Apply a linear gradient fill that fades to transparent at the bottom of the chart.

Animation Protocols

- All state transitions (score updates, view swaps) MUST use physical spring animations: `withAnimation(.spring(response: 0.5, dampingFraction: 0.7))`
- The fluid background should utilize iOS MeshGradients or a custom Metal shader for organic, lava-lamp-style color blending based on the current Stress Level Tier.

5. Copilot Implementation Plan (Step-by-Step Prompting Guide)

1. **Phase 1: Foundation.** Prompt Copilot to generate the SwiftData models for DailyMetric and StressReading. Set up the HealthKitManager class to handle read permissions for HRV, Sleep, and Heart Rate.
2. **Phase 2: Data Ingestion.** Prompt Copilot to implement the HKObserverQuery to run silently in the background. Ensure the app gracefully handles locked-device states (HealthKit data is encrypted and inaccessible when the iPhone is locked).
3. **Phase 3: The Math Engine.** Prompt Copilot to create the AlgorithmService. Have it feed dummy data to unit test the Z-score and rolling average mathematics to ensure the 0-100 scores output correctly before connecting it to live HealthKit streams.
4. **Phase 4: The Canvas & Animations.** Prompt Copilot to build the SwiftUI views starting with the static AreaMark charts. Once stable, prompt it to layer in the TimelineView wave animations, binding the animation speed variables to the published Stress Score.